

The Big Send-off: High-Performance Collectives on GPU-based Supercomputers

Siddharth Singh, Keshav Pradeep, Mahua Singh, Cunyang Wei, Abhinav Bhatele

Department of Computer Science, University of Maryland, College Park, USA

E-mail: ssingh37@umd.edu, bhatele@cs.umd.edu

Abstract—Collective communication is becoming increasingly important in supercomputer workloads with an increase in AI-related jobs. However, existing libraries that provide collective support such as NCCL, RCCL, and Cray-MPICH exhibit several performance and scalability limitations on modern GPU supercomputers. To address these challenges, we introduce the Performant Collective Communication Library (PCCL), specifically targeted for distributed deep learning (DL) workloads. PCCL provides highly optimized implementations of key collectives used in distributed DL: all-gather, reduce-scatter, and all-reduce. PCCL uses hierarchical algorithms and adaptive dispatching to scale efficiently to thousands of GPUs. It achieves substantial performance speedups over RCCL on 2048 GCDs of Frontier – up to $168\times$ for reduce-scatter, $33\times$ for all-gather and $10\times$ for all-reduce. More modest but still significant gains up to $5.7\times$ are observed on Perlmutter. These gains translate directly to end-to-end performance improvement for production DL workloads: up to $4.9\times$ speedup over RCCL in DeepSpeed ZeRO-3 training, and up to $2.4\times$ speedup over RCCL in DDP training.

Index Terms—collective communication, distributed deep learning, hierarchical collectives, all-reduce, all-gather, reduce-scatter, high-performance computing

I. MOTIVATION

Communication overheads in parallel applications become an increasing fraction of the overall execution time as these applications are scaled to more and more nodes and GPUs. In the case of deep learning (DL) applications [1], [2], [3], [4], [5], communication is typically composed of collective operations such as all-gather, all-reduce, and reduce-scatter. MPI and vendor-specific libraries such as NCCL from NVIDIA and RCCL from AMD provide implementations of such operations for use in DL frameworks. As opposed to traditional high-performance computing (HPC) applications, the message sizes in DL applications are significantly larger, from tens to hundreds of MBs, and current implementations of collective libraries struggle with maintaining high performance in this regime.

In this work, we analyze the efficiency and scaling limitations of existing communication libraries such as Cray-MPICH, NCCL and RCCL on multiple platforms. We examine three collectives that predominantly make up the communication in DL workloads: all-gather, reduce-scatter, and all-reduce. We discover that there are unique shortcomings, inefficiencies, and scalability issues in each of the messaging libraries. Primarily, we observe that all libraries struggle in

regimes of moderate message sizes (tens of MBs) and large GPU counts.

Figure 1 presents the results of benchmarking the all-gather operation on Frontier (AMD MI250X GPUs) and Perlmutter (NVIDIA A100 GPUs) with two output buffer (message) sizes: 64 and 128 MB using Cray-MPICH, NCCL, and RCCL. We observed that Cray-MPICH was the slowest on lower GPU/GCD counts but RCCL scaled the most poorly beyond hundreds of GCDs. The ideal scaling curve is a horizontal line and none of the libraries are able to achieve that. This suggests that there is a significant performance gap that can be closed, to improve the performance of large-scale DL workloads.

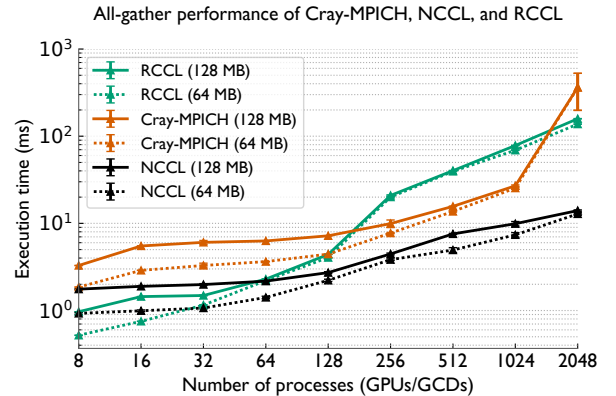


Fig. 1. Performance comparison of all-gather using RCCL (Frontier), Cray-MPICH (Frontier), NCCL (Perlmutter) for 64 and 128 MB output buffer sizes. The ideal scaling behavior (flat horizontal line) is not achieved by either library, highlighting their limited scalability at increasing GPU counts.

In this paper, we introduce the Performant Collective Communication Library (PCCL), designed to accelerate collective operations – specifically all-gather, reduce-scatter, and all-reduce – for large buffer sizes (>10 MB) found in parallel deep learning workloads. Our design exploits hierarchical algorithms, use of CUDA kernels for reductions, and adaptive selection between different choices to provide the best performance possible.

We evaluate PCCL on Frontier and Perlmutter, dragonfly-based supercomputers with AMD MI250X and NVIDIA A100 GPUs respectively, demonstrating its improved efficiency and scalability over existing communication libraries on both systems. Our implementations of all-gather, reduce-scatter,

and all-reduce achieve significant speedups over all three libraries – Cray-MPICH, NCCL, and RCCL. Our optimized collectives thus pave the way for scalable performance of high-performance, large-scale deep learning workloads on next-generation GPU supercomputers.

The paper makes the following key contributions:

- We analyze the limitations of existing communication libraries – Cray-MPICH, NCCL, and RCCL on Perlmutter and Frontier for all-gather and reduce-scatter operations in parallel deep learning workloads.
- We develop optimized implementations of all-gather, reduce-scatter, and all-reduce in PCCL, with a focus on effectively utilizing system resources and ensuring scalability for large messages and GPU counts.
- We demonstrate substantial performance speedups over RCCL on 2048 GCDs of Frontier – up to $168\times$ for reduce-scatter, $33\times$ for all-gather and $10\times$ for all-reduce. More modest but still significant gains up to $5.7\times$ are observed on Perlmutter.
- We conduct end-to-end benchmarking of large-scale, multi-billion-parameter LLM training workloads to validate the practical benefits of our optimizations, demonstrating significant speedups in training throughput: up to $4.9\times$ speedup over RCCL in DeepSpeed ZeRO-3 training, and up to $2.4\times$ speedup over RCCL in DDP training.

II. BACKGROUND ON COLLECTIVE OPERATIONS

In this section, we provide relevant background on the role of collective communication in distributed deep learning workloads.

A. Collective Communication in Distributed Deep Learning

Sharded data parallelism (SDP) and distributed data parallelism (DDP) are widely used for large scale training [2], [1], [6]. They are briefly described below.

Collective Communication in Sharded Data Parallelism:

In SDP, Model parameters and gradients are partitioned (or “sharded”) across GPUs, which significantly reduces memory requirements and allows for the training of extremely large models. Two critical collective communication operations – *all-gather* and *reduce-scatter* – play a central role. These operations aggregate distributed data across GPUs: the all-gather operation collects model parameters from all shards to form a complete copy, while the reduce-scatter operation performs a reduction and distributes gradients across participating processes. In Figure 2, we plot the all-gather and reduce-scatter message sizes for three frameworks that support sharded data parallelism – FSDP [2], Deepspeed ZeRO-3 [1], and AxoNN¹ [7]. Notice how the message sizes across these three frameworks are in the tens to hundreds of megabytes, even becoming more than a gigabyte for larger models.

¹Unlike FSDP and ZeRO-3, AxoNN performs all-gathers and reduce-scatters for each linear layer separately, which results in a wide range of buffer sizes.

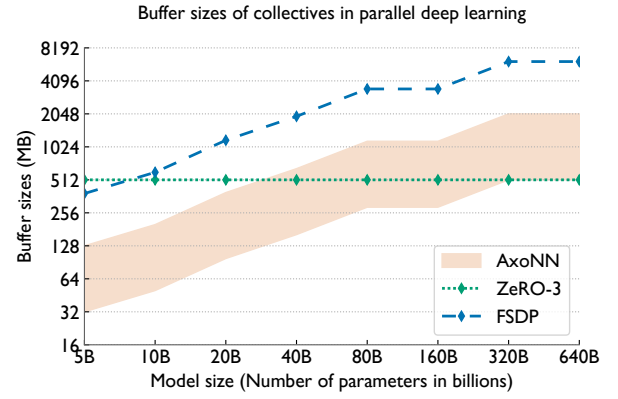


Fig. 2. Distribution of all-gather and reduce-scatter message sizes for several deep learning frameworks for a range of transformer [8] model sizes. The y-axis represents input buffer sizes for all-gathers but output buffer sizes for reduce-scatters.

Collective Communication in Distributed Data Parallelism:

In DDP, model parameters are replicated across GPUs, and the *all-reduce* collective is critical for synchronizing parameter replicas. During training, each GPU performs forward and backward passes on a distinct local minibatch of the input dataset, producing gradients specific to each GPU’s minibatch. After the backwards pass, DDP invokes all-reduce to average the gradients across GPUs, and each GPU performs a local parameter update to keep all replicas in sync. Due to the unique communication pattern of DDP, the all-reduce message sizes are exactly the model size. For example, training a 1B-parameter model in FP32, requires exchanging 4 GB of gradients per iteration. To reduce latency and improve overlap with computation, PyTorch’s DDP framework splits this large all-reduce into several smaller all-reduces with sizes ranging from 48–80 MB [4].

B. Algorithms for Dissolving Different Collectives

Efficient implementations of all-gather, reduce-scatter, and all-reduce operations are critical for distributed deep learning. In this work, we build on several well-established algorithms and introduce enhancements to improve performance and scalability.

Ring: The ring algorithm is a popular method for implementing collective communications due to its simplicity and efficiency in certain network topologies. In a ring-based collective operation, each process communicates with its immediate neighbors in a circular fashion. While effective at moderate scales and large message sizes, the ring algorithm can suffer from inefficiencies at larger scales due to its latency term being linearly proportional to the number of processes. The communication time of a ring all-gather can be modeled as,

$$T_{\text{ring}} = \underbrace{\alpha}_{\text{startup latency}} \times \underbrace{(p-1)}_{\text{number of processes}} + \underbrace{\beta}_{\text{inverse of bandwidth}} \times \frac{p-1}{p} \times \underbrace{m}_{\text{buffer size}} \quad (1)$$

where p is the number of processes, α represents the startup latency per message, m is the size of the input buffer on each GPU, and β is the inverse of the peer-to-peer bandwidth.

Recursive Halving/Doubling: A popular way of minimizing latency costs involves utilizing the recursive halving or doubling algorithms for all-gathers and reduce-scatters respectively. These algorithms divide the communication task into logarithmically many steps, and hence their performance scales better than the ring algorithm. The communication time for a recursive-doubling all-gather can be modeled as,

$$T_{\text{rec}} = \underbrace{\alpha}_{\text{startup latency}} \times \log_2(\underbrace{p}_{\text{number of processes}}) + \underbrace{\beta}_{\text{inverse of bandwidth}} \times \frac{p-1}{p} \underbrace{m}_{\text{buffer size}} \quad (2)$$

For small message sizes or very large process counts, the logarithmic growth in the latency term often leads to lower overall communication costs. Building on these primitives, an all-reduce can be implemented as a reduce-scatter followed by an all-gather operation. More details about these and other algorithms can be found in Thakur et al. [9].

III. CURRENT STATE OF COMMUNICATION LIBRARIES

This section investigates the current state of popular communication libraries - Cray-MPICH, NCCL, and RCCL, on Perlmutter and Frontier. We find unique issues that limit the performance of each library, and highlight these below via experiments.

A. Benchmarking Setup

To ensure a fair comparison, we follow best practices for benchmarking all three libraries – optimal process placement, NUMA-aware NIC binding, disabling eager messaging, and enabling GPU Direct RDMA [10].

Message Sizes and Measurement Protocol: In line with Figure 2, our evaluation in this section focuses on two message sizes – 256 MB and 512 MB. Note that for all-gathers and reduce-scatters, these values refer to the output and input message size per GPU respectively. For each combination of library, collective, message size, and GPU count, we perform ten independent trials. We measure the total time spent in the collective during each run using AMD’s `hipeventtimers` instrumentation on Frontier, and NVIDIA’s `cudaEventElapsedTime` on Perlmutter. All reported timings are end-to-end measurements that include any necessary data movement and transformation costs, including intra-GPU transposes required by hierarchical algorithms. We compute the mean and standard deviation over the ten trials to ensure statistical robustness.

Software Stack: On Frontier, our software stack comprises of ROCm 6.2.4, RCCL 2.20.5, Cray-MPICH 8.1.31 distribution, libfabric 1.15.2 and the aws-ofi-rccl plugin version v1.4. On Perlmutter, we use CUDA 12.4, NCCL 2.24.3, and Cray-MPICH 8.1.30, and libfabric 1.22.0.

B. Resource Under-utilization in Cray-MPICH

Figure 3 (left) presents a comparative analysis of all-gather performance between Cray-MPICH and RCCL on Frontier for message sizes of 256 MB and 512 MB. We observe that RCCL achieves approximately a 4× performance advantage in this bandwidth-bound scenario. To explain this disparity, we examine two hardware performance counters provided by the Cassini Slingshot-11 Network Interface Controllers (NICs) [11] - `parbs_tarb_pi_posted_pkts` and `parbs_tarb_pi_non_posted_pkts`. These counters represent the count of packets read from and written to each NIC within a node during job execution, respectively. As shown in Figure 3 (middle, right), we observe a consistent trend across all runs – Cray-MPICH exclusively utilizes NIC-0 for all write operations, and NIC-3 for all read operations. Conversely, RCCL distributes its read and write operations evenly across all four NICs on the node. This underutilization of available network resources by Cray-MPICH explains the 4× performance gap we observe in all-gather operations.

Moreover, for collectives that require computation, such as reduce-scatters and all-reduces, Cray-MPICH suffers from another performance bottleneck – it performs the reduction operations on the CPU instead of offloading them to the GPU. This introduces significant computational overheads, as shown in Figure 4. Cray-MPICH (orange) performs significantly worse than RCCL (green). Notably, this performance gap is far greater than the 4× difference observed earlier for all-gather in Figure 3.

While the NIC underutilization issue outlined previously still persists for Cray-MPICH reduce-scatter operations, it alone cannot explain this performance disparity. We hypothesize that this disparity stems from Cray-MPICH’s choice to perform reductions on the CPU instead of the more powerful GPUs. To validate this, we manually implemented the reduce-scatter operation using MPI point-to-point primitives and a GPU vector addition kernel. As shown in Figure 4, our implementation (blue line) achieves performance that is several times faster than Cray-MPICH’s native reduce-scatter, further supporting our hypothesis.

Observation 1

Cray-MPICH severely underutilizes available network (NIC) and computational (GPU) resources. It routes all network traffic through a single NIC, and performs reduction operations on the CPU, instead of offloading them to the GPU.

C. Poor Scaling of NCCL and RCCL at Large GPU Counts

Performance limitations of Cray-MPICH have made NCCL and RCCL popular for distributed deep learning, but as we demonstrate next, these libraries have algorithmic limitations that prevent them from scaling efficiently to large GPU counts. Let us again look at all-gather performance of RCCL (Frontier) and NCCL (Perlmutter) in Figure 1. We observe that both of these libraries scale extremely poorly. On investigating

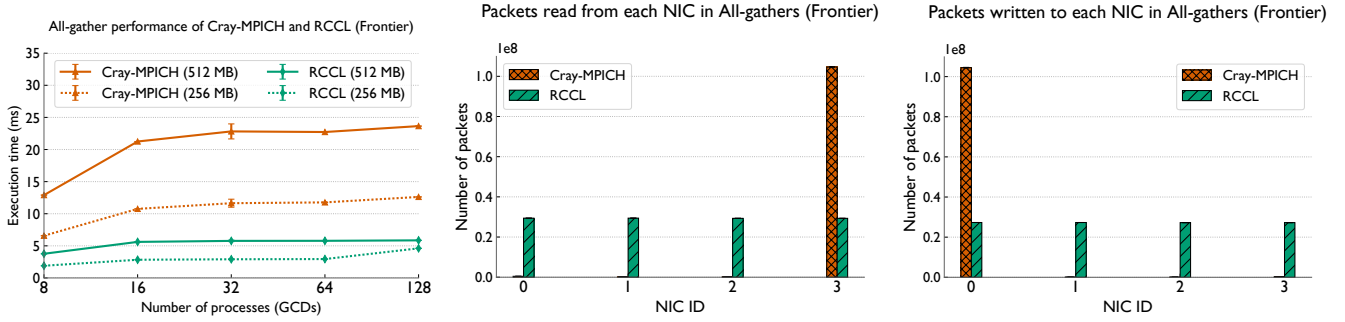


Fig. 3. All-gather performance of Cray-MPICH and RCCL on Frontier for a bandwidth-bound scenario with large message sizes (256 and 512 MB) and small GPU counts (left). The middle and right plots demonstrate the number of packets read from (middle) and written to (right) each of the four NICs on a Frontier compute node during all-gather operations.

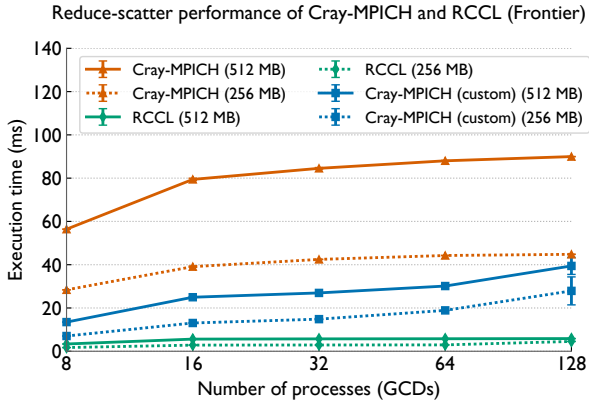


Fig. 4. Performance comparison of reduce-scatter using Cray-MPICH, RCCL, and a custom implementation of reduce-scatter that uses MPI point-to-point primitives and GPU compute kernels.

deeper, we found that NCCL and RCCL only support the ring algorithm for all-gather and reduce-scatter (refer to Section II-B). While effective for bandwidth-bound workloads, the ring algorithm performs poorly in latency-sensitive scenarios because each process must send and receive $(p - 1)$ messages sequentially, causing the total communication time to grow linearly with the number of processes.

Notably, while NCCL and RCCL implement a logarithmic latency scaling algorithm for all-reduce based on the double-binary tree structure [12], for all-gather and reduce-scatter, they lack log latency scaling algorithms such as recursive doubling or halving (refer to Section II-B)². These are known to reduce the number of communication steps to $\log_2 p$ and are generally preferred for small message sizes or high process counts. This lack of algorithmic diversity directly contributes to the sub-optimal scaling we observe at large GPU counts.

Observation 2

NCCL and RCCL rely solely on the ring algorithm for all-gather and reduce-scatter, leading to poor scaling in latency-bound scenarios. More efficient algorithms such as recursive doubling and halving are not supported.

IV. DESIGN OF SCALABLE COLLECTIVES IN PCCL

In Section III, we identified several challenges affecting Cray-MPICH, NCCL, and RCCL, in the context of all-gather and reduce-scatter collectives for deep learning workloads. These challenges create significant barriers to efficiently scaling DL workloads across thousands of GPUs. We now present our new designs of these collectives, implemented in a new library called the Performant Collective Communication Library or PCCL in short. PCCL provides two inter-node algorithm backends – PCCL_ring (ring-based) and PCCL_rec (recursive doubling/halving) – along with an ML-guided adaptive dispatcher that selects the best-performing backend at runtime.

A. Hierarchical Collective Algorithms for Scalability

Our optimized implementations of all-gather, reduce-scatter, and all-reduce are based on a two-level hierarchical design. While our primary motivation is to address the underutilization of NICs (identified in Section III-B), this design also reduces latency and improves scalability [14], [15].

We illustrate our design in Figure 5 for an all-gather operation on a hypothetical system with N nodes and M GPUs per node. The global collective operation to be performed across all GPUs is divided into two distinct phases (inter-node and intra-node) using sub-communicators. Inter-node sub-communicators are formed by grouping together corresponding GPUs across nodes. For example, in Figure 5, all GPUs with the same within-node index are grouped together to create a total of M inter-node sub-communicators. Similarly, intra-node sub-communicators are formed by grouping

²NCCL has recently introduced a logarithmic scaling algorithm for all-gather and reduce-scatter called PAT; however, at the time of writing, it only supports one GPU per node [13].

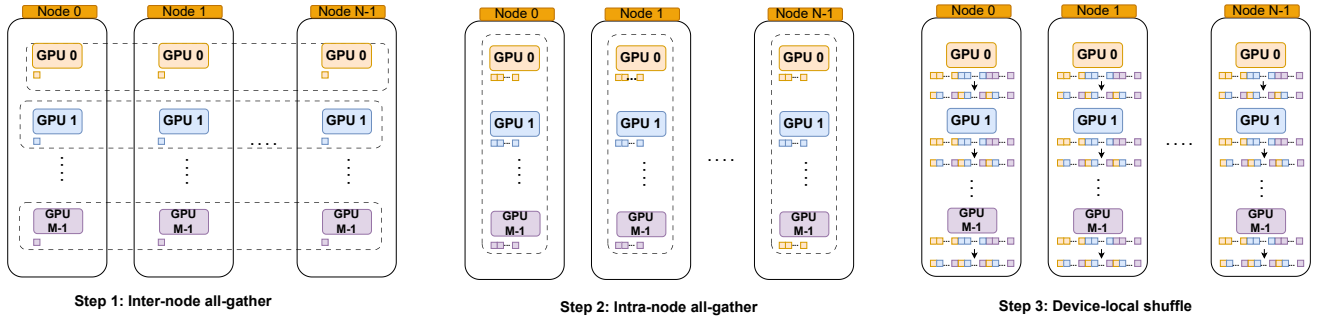


Fig. 5. Diagram showing our hierarchical (two-level) implementation to dissolve an all-gather operation on a GPU-based cluster with N nodes and M GPUs per node. In step 1, we perform inter-node all-gathers, in step 2, we perform intra-node all-gathers, and in step 3, each GPU performs a local shuffle of the received data.

together all GPUs within a node to create N intra-node sub-communicators.

The hierarchical communication for dissolving the collective unfolds in three steps. First, we schedule concurrent all-gather operations within all inter-node sub-communicators (Step 1 of Figure 5). After this phase, each GPU has received data from its counterpart in every other node, yielding a partial result distributed across GPUs within each node. Second, we perform an intra-node all-gather (Step 2 of Figure 5), after which each GPU holds the complete output in memory, albeit in an incorrect order. Third, a device-local shuffle (Step 3 of Figure 5) rearranges each GPU’s data into the correct order; in practice, this is implemented as a transpose kernel. We implement reduce-scatter similarly, starting with the intra-node phase followed by the inter-node phase. All-reduce is implemented by composing the two-level reduce-scatter with the two-level all-gather.

This design addresses NIC under-utilization by scheduling all inter-node all-gathers in Step 1 concurrently. On Frontier, each node has four NICs connected to two GCDs each. We ensure each GCD exclusively uses its corresponding NIC (e.g., GCDs 0 and 1 use NIC 0, GCDs 2 and 3 use NIC 1, etc.), evenly distributing inter-node traffic across all NICs. While our hierarchical design explicitly exploits the intra-node topology by mapping GPUs to their respective NICs, we do not explicitly map our communication patterns to the inter-node network topology. On Dragonfly-based systems such as Frontier and Perlmutter, UGAL routing distributes inter-node traffic across the network, reducing the need for topology-aware collective mapping.

Choice of Communication Libraries: Our choice of libraries for each level of the hierarchy is driven by performance and reliability. For intra-node communication, we use GPU-vendor libraries (NCCL or RCCL) as they are highly optimized for intra-node topologies, efficiently utilizing shared memory, PCIe, and Infinity Fabric or NVLink connections [10]. NCCL and RCCL only support the ring algorithm for intra-node all-gather and reduce-scatter. Since the ring algorithm is well-suited when the number of GCDs/GPUs per node is small (eight on Frontier, and four on Perlmutter), we adopt this for

all intra-node collectives. This ensures that the GPU-GPU link bandwidth is saturated effectively.

For inter-node communication, we use MPI due to RCCL failures at scale, as reported in prior work [16] and noted by HPE³ and the OLCF User Guide⁴. Moreover, we find that Cray-MPICH exhibits lower performance variability than RCCL on Slingshot interconnects, further motivating its use at the inter-node level.

Messaging Protocol Selection: For intra-node communication, we rely on the vendor libraries’ internal protocol selection heuristics, such as NCCL and RCCL’s ability to switch to low-latency (LL/LL128) protocols for small messages on NVLink and Infinity Fabric. While Cray-MPICH does not support these low-latency protocols for inter-node communication, this is not a concern as the scope of our work is focused on large message sizes (16 MB to 1 GB).

B. Implementing Custom Algorithms for Inter-node Operations

The use of MPI for the collective operations in the inter-node phase presents some challenges. With potentially thousands of GPUs participating in the collective, the choice of communication algorithm becomes critical for performance. However, Cray-MPICH only offers the ring algorithm. As described in Section II-B, the ring algorithm’s linear scaling in latency with respect to the number of processes makes it suboptimal at large-scale.

As a result, we have developed custom implementations of each collective in PCCL for the inter-node phase that use MPI point-to-point send and receive messages. We implement two different algorithms as described below.

Ring Algorithm: We implement an inter-node version of the ring algorithm described in Section II-B. This can be more performant in bandwidth-bound scenarios. We refer to our implementation using the ring algorithm as `PCCL_ring`.

³https://www.olcf.ornl.gov/wp-content/uploads/OLCF_AI_Training_0417_2024.pdf

⁴https://docs.olcf.ornl.gov/software/analytics/pytorch_frontier.html#environment-variables

Recursive Doubling/Halving Algorithm: We utilize the recursive doubling algorithm for inter-node all-gather and recursive halving for inter-node reduce-scatter operations [9], and implement them using MPI point-to-point messages. These algorithms offer logarithmic latency terms (refer to Section II-B), enabling significantly better performance as the number of GPUs increases. Moreover, for reduce-scatter operations, we also ensure that our vector addition computation is efficient by scheduling it on GPU cores. We refer to our implementation using recursive doubling/halving as `PCCL_rec`. Further, our all-reduce in `PCCL_rec` uses recursive halving followed by recursive doubling for inter-node traffic.

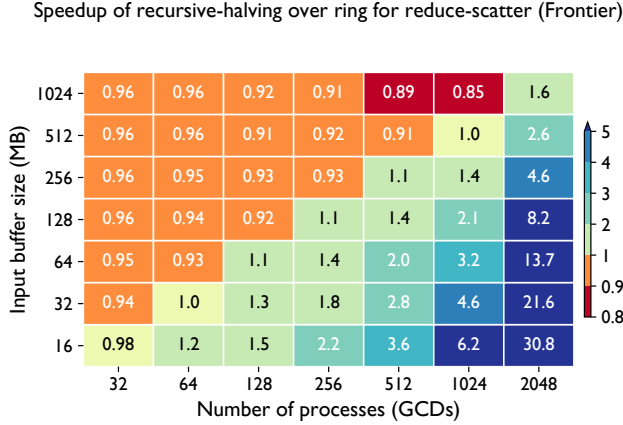


Fig. 6. Heatmap showing speedups from using recursive halving over the ring algorithm in the inter-node phase of the reduce-scatter implementation in PCCL

Figure 6 shows the speedup of recursive halving over ring for inter-node reduce-scatter. In bandwidth-bound scenarios (fewer processes and/or larger messages), ring performs better. In latency-bound scenarios (more processes and/or smaller messages), recursive halving achieves significant improvements, confirming our algorithmic complexity analysis. Our goal is to be able to choose the ideal algorithm in different scenarios, and we describe our efforts in that direction below.

We have developed these implementations in C++ as part of PCCL and expose Pybind11 bindings to enable seamless integration with Python-based deep learning frameworks such as ZeRO-3 [1]. Implementing these algorithms in C++ proves to be critical for achieving high performance.

C. ML-guided Adaptive Dispatching

We have observed, based on our empirical analysis, that no single communication library – Cray-MPICH, NCCL, RCCL, `PCCL_ring`, or `PCCL_rec` – is universally best. Performance depends on the specific configuration, with message size and GPU count being the dominant factors. For example, RCCL often outperforms PCCL’s ring/recursive algorithms at lower GPU counts and larger message sizes. This motivated us to develop an adaptive dispatcher capable of selecting the most suitable library at runtime. This modified design of the PCCL library is shown in Figure 7.

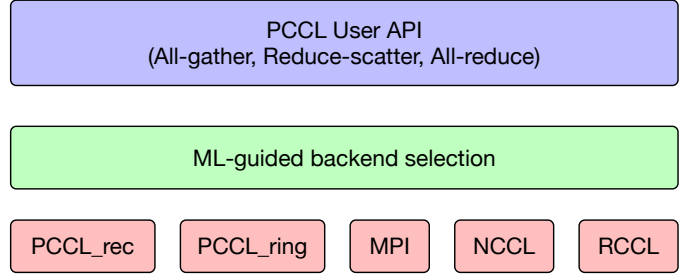


Fig. 7. The ML-guided selection mechanism in PCCL can enable choosing the best performing backend from several available options.

In order to enable optimal backend selection, we implemented a lightweight dispatcher based on Support Vector Machines (SVMs) – a supervised learning algorithm that classifies data by finding the optimal decision boundary that separates distinct categories. SVMs work by identifying the hyperplane that maximizes the margin between different classes in the feature space, allowing for effective classification even in complex scenarios. For each (machine, collective) pair, we train a dedicated SVM classifier using empirical data spanning message sizes from 1 MB to 1024 MB and GPU counts from 4 to 2048. For each configuration (library, message size, GPU count), we include ten independent runs in our dataset. This dataset is partitioned using a stratified 80/20 train-test split to maintain class balance. Hyperparameter selection for each SVM is performed via five-fold cross-validation on the training set, ensuring robust model selection and mitigating overfitting. At runtime, the dispatcher queries the appropriate SVM with the GPU count and message size as input features to predict the optimal backend. We evaluate the performance of the trained model on 20% unseen test data. As Table I shows, the high prediction accuracy and low misclassification rates indicate that the dispatcher generalizes well to previously unseen configurations.

TABLE I
SVM DISPATCHER PERFORMANCE ON THE HELD-OUT TEST SET (20% OF DATA). ACCURACY IS GIVEN IN PERCENT.

Machine	Collective	Test Size	Correctly Classified	Accuracy
Frontier	All-Gather	20	17	85.0
	Reduce-Scatter	20	18	90.0
	All-Reduce	20	16	80.0
Perlmutter	All-Gather	22	20	90.9
	Reduce-Scatter	22	21	95.4
	All-Reduce	20	15	75.0

It is important to note that these SVM models are trained per system, per collective and must be retrained when deploying PCCL on different machines. Our feature set includes only GPU count and message size; we do not explicitly encode node allocation or physical placement as features. Instead, the 10-run averaging strategy captures performance variations arising

from different allocations. On both Frontier and Perlmutter, the underlying Dragonfly network topology with UGAL (Universal Globally Adaptive Load-balancing) routing [17] provides inherent robustness to job placement, as UGAL dynamically selects minimal-load paths. However, on systems with different topologies or routing schemes, physical placement may have a more pronounced impact on performance, and incorporating topology-aware features could further improve model accuracy.

V. EXPERIMENTAL SETUP

Next, we provide details of the experimental setup for our comparison of PCCL-based collectives with other communication libraries, and in the context of production DL workloads.

A. Benchmarking Collective Operations

First, we compare the standalone performance of all-gather, reduce-scatter, and all-reduce operations, which are the primary focus of this paper. Our experiments cover a range of message sizes from 16 MB to 1 GB. For all-gather, this range denotes each GPU’s output buffer size; for reduce-scatter, the input buffer size; for all-reduce, both input and output buffer sizes. For each message size, we measure performance across 32 to 2048 GCDs (4 to 256 nodes) on Frontier, and 32 to 2048 GPUs (8 to 512 nodes) on Perlmutter.

We conduct ten independent trials for each combination of library, collective, message size, and GPU count. We consider the following libraries on each system: Cray-MPICH (the default MPI implementation on HPE Cray systems with Sling-shot interconnects), NCCL or RCCL, and PCCL. For both systems, our setup for process placement, measurement protocol, communication tuning, and software stack is consistent with the configuration described in Section III-A. For all-reduce, at the time of our runs, a more recent software stack was available: on Frontier we use ROCm 6.4.1, RCCL 2.22.3, Cray MPICH 8.1.32 distribution, libfabric 1.22.0 and the aws-ofi-rccl plugin version v1.4, while on Perlmutter, we use CUDA 12.9, NCCL 2.27.3, and Cray-MPICH 8.1.30, and libfabric 1.22.0.

B. Comparative Evaluation of Production DL Workloads

DeepSpeed ZeRO-3: To evaluate the practical benefits of PCCL’s all-gather and reduce-scatter, we measure the end-to-end training performance of large language models using DeepSpeed ZeRO-3 [1], a widely adopted parallel deep learning framework. We perform strong scaling experiments on seven billion and 13 billion parameter GPT-style transformer models [18] using model hyperparameters from Zhang et al. [19]. We list these hyperparameters in Table II. We use a global batch size of four million tokens and a sequence length of 2048 and use the OpenWebText [20] corpus to create our training data.

TABLE II
ARCHITECTURAL DETAILS OF THE GPT-STYLE TRANSFORMER MODELS [18] USED IN THE EXPERIMENTS. WE BORROW THESE HYPERPARAMETERS FROM ZHANG ET AL. [19].

Model	Framework	Params	Layers	Hidden Size	Heads
GPT-7B	ZeRO-3	7B	32	4096	32
GPT-13B	ZeRO-3	13B	40	5120	40
GPT-1.3B	DDP	1.3B	24	2048	32

PyTorch DDP: PyTorch DDP is a distributed data parallel framework which relies on all-reduce (unlike ZeRO-3). To evaluate the practical benefits of PCCL’s all-reduce, we use PyTorch DDP [4] to train a 1.3 billion parameter GPT-style transformer model [18], with hyperparameters summarized in Table II. We use a global batch size of one million tokens and use the OpenWebText [20] corpus as training data.

For both ZeRO-3 and DDP experiments, on Frontier, we scale from 128 to 1024 GCDs, and on Perlmutter, we scale from 256 to 2048 GPUs. We run experiments with RCCL (Frontier) and NCCL (Perlmutter) as baselines, then swap in PCCL_rec to handle all-gather and reduce-scatter in ZeRO-3, and to handle all-reduce in DDP. For each configuration, we run 10 training batches across three trials and compute the average time per iteration over the last eight batches in each run to minimize warm-up effects. We observed significant variability in RCCL all-reduce performance; to account for this, we ran five trials for DDP experiments and report results from the three trials with the smallest mean batch time.

VI. PERFORMANCE RESULTS

We begin with examining the performance of PCCL collectives using adaptive dispatching (described in Section IV-C) and comparing it against other state-of-the-art communication libraries.

A. Comparisons with Cray-MPICH and NCCL on Perlmutter

Figure 8 presents results for all-gather (left) and reduce-scatter (middle) collectives on Perlmutter for representative per-process message sizes of 256 and 512 MB. For all-gather and reduce-scatter, we observed that both Cray-MPICH (orange lines) and NCCL (black lines) fell short of ideal scaling, which should be a flat horizontal line. NCCL’s performance begins to degrade noticeably beyond 512 processes. In contrast, PCCL scales nearly perfectly across both collectives, maintaining desirable performance curves, and achieving speedups in the range of $1.3 - 4.6\times$ over NCCL and $8.8 - 15\times$ over Cray-MPICH on 1024 and 2048 GPUs. Figure 8 (right) presents results for all-reduce operations with message sizes of 64 MB and 128 MB. Both NCCL and PCCL exhibit strong scalability with node count. NCCL employs double-binary trees to achieve log-latency scaling, which explains why the performance of NCCL and PCCL is nearly identical for all-reduce.

Figure 9 presents heatmaps that present the relative performance improvement of PCCL over NCCL for a larger range

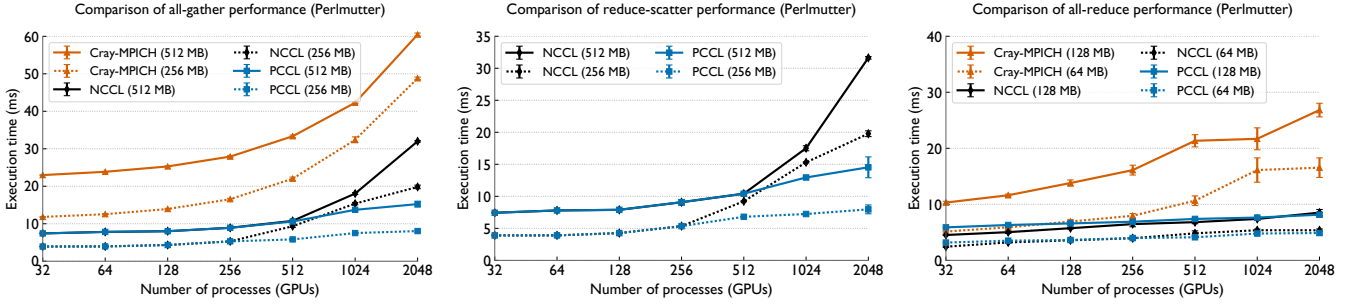


Fig. 8. Performance comparison of all-gather (left), reduce-scatter (middle), and all-reduce (right) using Cray-MPICH, NCCL, and PCCL with adaptive dispatching, for different per-process buffer sizes (256 and 512 MB for all-gather and reduce-scatter; 64 and 128 MB for all-reduce) and varying process counts on Perlmutter.

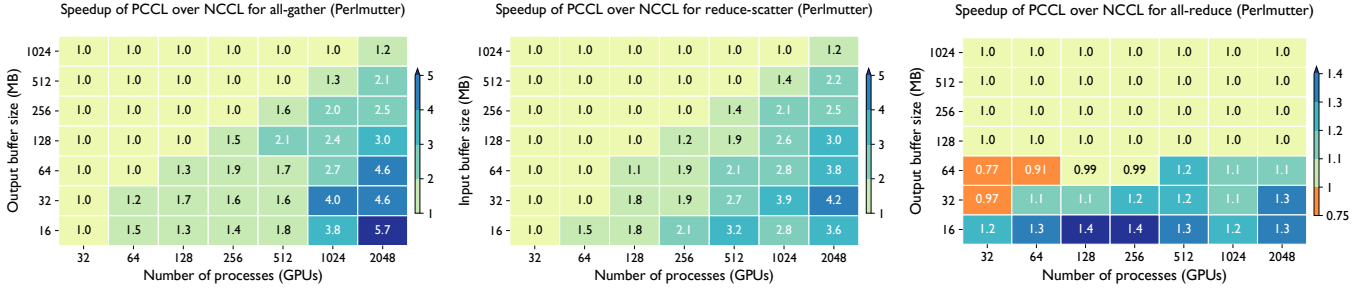


Fig. 9. Heatmaps showing speedups from using PCCL with adaptive dispatching over NCCL for all-gather (left), reduce-scatter (middle), and all-reduce (right) on Perlmutter. The speedup is shown as a function of per-process output/input buffer size (in MB) and process count.

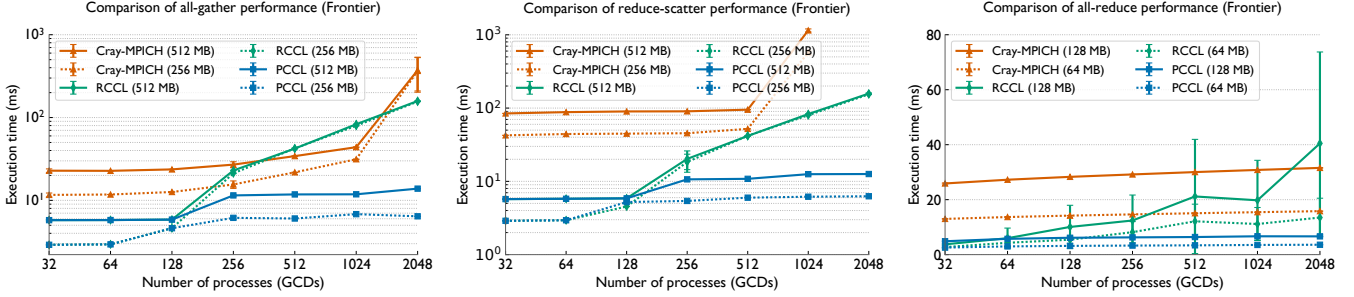


Fig. 10. Performance comparison of all-gather (left), reduce-scatter (middle), and all-reduce (right) using Cray-MPICH, RCCL, and PCCL with adaptive dispatching, for different per-process buffer sizes (256 and 512 MB for all-gather and reduce-scatter; 64 and 128 MB for all-reduce) and varying process counts on Frontier.

of message sizes and GPU counts. For all-gather (left) and reduce-scatter (middle), PCCL performs similar to NCCL in the top-left regions of the heatmaps, representing bandwidth-bound scenarios. As expected, our adaptive dispatching protocol selects NCCL for these cells. However, as we transition to latency-sensitive regions in the bottom-right corners of the heatmap, PCCL’s advantages become evident. Around 1024–2048 processes and 16–32MB message sizes, PCCL achieves significant speedups over NCCL, ranging from 3–5 $\times$. While speedups for larger message sizes are smaller, they remain notable. For example, at 2048 processes and 128–512 MB message sizes, PCCL is approximately 2–3 $\times$ faster than NCCL, which is still a significant improvement. These results highlight PCCL’s effectiveness in accelerating collective com-

munication for parallel deep learning – across both extreme scales and diverse GPU architectures. For all-reduce (right), we observed similar performance between PCCL and NCCL as both libraries use log-latency scaling algorithms.

B. Comparisons with Cray-MPICH and RCCL on Frontier

Next, we evaluate PCCL’s effectiveness on Frontier, which features AMD MI250X GPUs. Figure 10 (left) shows the performance of all-gather operations on Frontier using PCCL and other communication libraries for output buffer sizes 256 and 512 MB. For each configuration, we scale the number of GCDs from 32 to 2048. Since the output buffer size per GPU remains fixed, the ideal performance curve for each buffer size is a flat horizontal line, indicating perfect scaling.

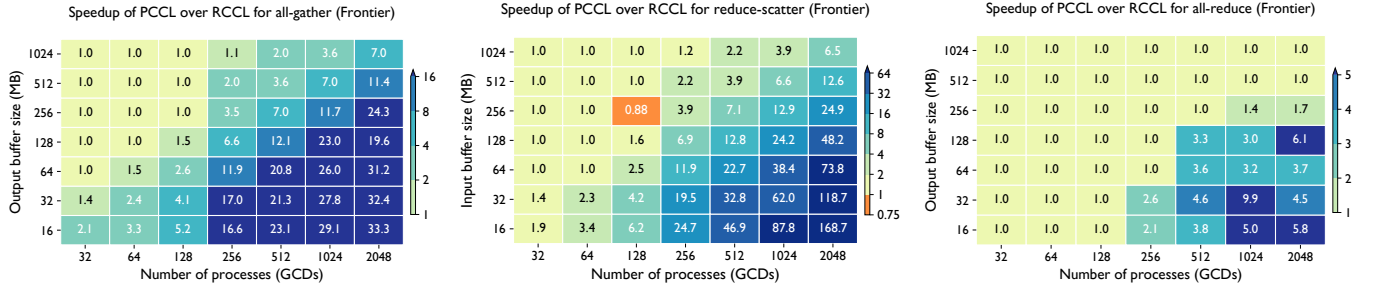


Fig. 11. Heatmaps showing speedups from using PCCL with adaptive dispatching over RCCL for all-gather (left), reduce-scatter (middle), and all-reduce (right) on Frontier. The speedup is shown as a function of per-process output/input buffer size (in MB) and process count.

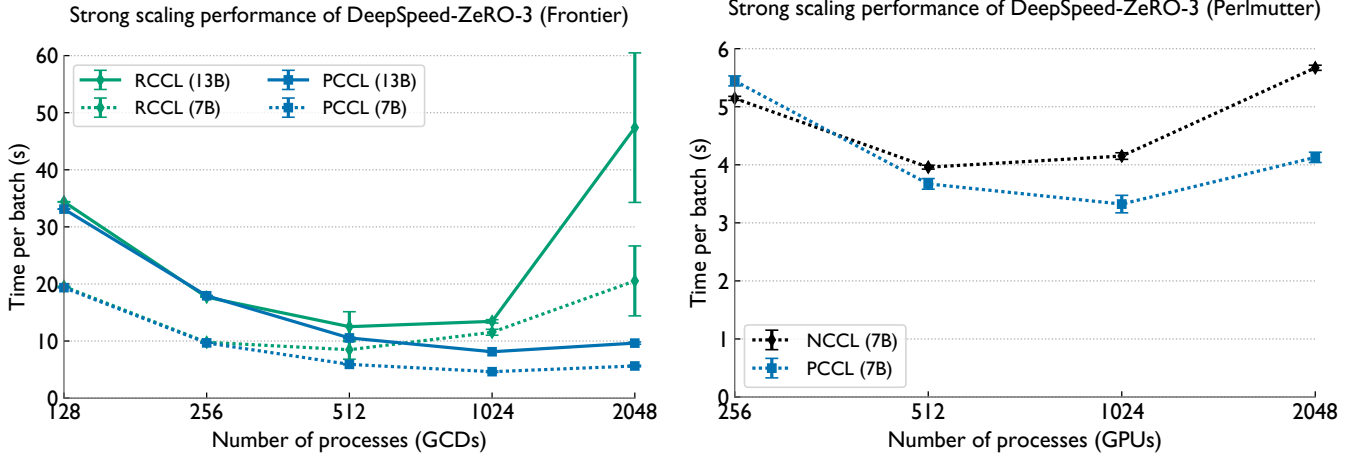


Fig. 12. Strong scaling performance of DeepSpeed ZeRO-3 using RCCL or NCCL, and PCCL, on Frontier (left) and Perlmutter (right) for two model sizes: GPT-3 7B and GPT-3 13B.

We observed that RCCL and Cray-MPICH fell short of the ideal scaling line. Beyond 128 GCDs, RCCL’s (green) execution time scaled almost linearly with the number of processes. Cray-MPICH (orange lines) showed a similar pattern with performance dropping sharply as we scaled to higher process counts. We attribute the poor scaling of RCCL and Cray-MPICH to their reliance on the ring algorithm, whose latency grows linearly with the number of processes.

In contrast, PCCL (blue) maintains nearly flat scaling trends across all message sizes, demonstrating significantly better scalability and efficiency. We attribute PCCL’s better performance to its hierarchical algorithms, described in Section IV. By using the ring algorithm within nodes (limited to eight processes) and recursive doubling across nodes, PCCL bounds the latency overhead that otherwise grows linearly in traditional ring-based implementations. This design enables better scalability across large GPU counts. The performance improvements of PCCL over RCCL and Cray-MPICH become increasingly pronounced as we increase the number of processes. At 2048 GCDs, PCCL all-gather achieves speedups ranging from 7–24 $\times$ over RCCL, and an even larger 27–82 $\times$ over Cray-MPICH, depending on the message size. These results highlight PCCL’s ability to deliver high-performance communication at scale.

The performance trends for reduce-scatter on Frontier follow the same pattern established by all-gather, as depicted in Figure 10 (middle). Both RCCL and Cray-MPICH fall short of the ideal performance curve, and PCCL achieves significant speedups over both libraries with increasing scale. The results differ for all-reduce operations, however, as shown in Figure 10 (right). Here, RCCL employs double-binary trees to achieve log-latency scaling, which explains its improved scalability over its own reduce-scatter or all-gather algorithms. Despite this, PCCL’s two-phase strategy demonstrates near-flat scaling, ultimately achieving significant speedups over RCCL at higher GCD counts.

Figure 11 shows the speedups of PCCL over RCCL for all-gather (left), reduce-scatter (middle), and all-reduce (right) operations on Frontier, respectively, across a range of output buffer sizes and process counts. In the top-left regions of all three heatmaps, where we have large messages and small GPU counts, representing bandwidth-bound scenarios, PCCL’s adaptive dispatching picks RCCL as the backend. This is expected, as RCCL’s flat ring algorithm can achieve higher bandwidth than PCCL’s hierarchical two-phase strategy [14].

In contrast, in the bottom-right corners of the heatmap, representing the latency-sensitive regime with small messages and large GPU counts, PCCL delivers substantial gains. For

larger message sizes such as 512 MB and 1024 MB, the speedups are relatively smaller but still significantly high: 11.4 and 7 $\times$ for all-gather, and 11.4 and 6.2 $\times$ for reduce-scatter, respectively. On 2048 GCDs, for 16, 32, and 64 MB buffer sizes, PCCL achieves speedups of more than 30 $\times$ and 50–100 $\times$ over RCCL for all-gather and reduce-scatter, respectively. To understand the source of these large speedups, we analyzed Cassini NIC counter values for RCCL and PCCL at 2048 GCDs. Two counters stand out: RCCL exhibits 200 $\times$ higher `lpe_net_match_overflow_0` and 120 $\times$ higher `lpe_rdzv_puts_0` compared to PCCL. These counters indicate dramatically higher pressure on the NIC’s matching tables and significantly more rendezvous PUT traffic, respectively, both of which may contribute to RCCL’s observed performance degradation at scale. For larger messages at 2048 GCDs, the speedups are comparatively smaller but still significant—24.3 and 7.0 $\times$ for all-gather, 24.9 and 6.5 $\times$ for reduce-scatter on 256 and 1024 MB, respectively. Similarly, for all-reduce, PCCL achieves up to a 9.9 $\times$ speedup over RCCL in regions where latency dominates. These results underscore PCCL’s strength in latency-bound scenarios and highlight its ability to scale efficiently to thousands of GPUs.

C. Impact on DL Training Performance

Finally, we examine how these communication gains translate into improvements in end-to-end training performance at scale. Figure 12 (left) presents the batch times for strong scaling GPT-3-style transformer training on Frontier using the DeepSpeed ZeRO-3 framework [1]. Green lines represent ZeRO-3 runs with RCCL, the default communication library and blue lines represent runs with all-gather and reduce-scatter collectives in ZeRO-3 issued with PCCL. At smaller scales (128 and 256 GCDs), both libraries perform comparably. However, as we scale further, PCCL begins to outperform RCCL. When scaling to 1024 GCDs, RCCL fails to maintain strong scaling and even exhibits increased batch times compared to 512 GCDs. In contrast, PCCL continues to scale efficiently, delivering a 2.5 $\times$ speedup for the 7B model and a 1.6 $\times$ speedup for the 13B model. Finally, at 2048 GCDs, although both libraries exhibit diminishing returns in strong scaling efficiency, PCCL still achieves substantial speedups 3.3–4.9 $\times$ relative to RCCL.

We observed similar trends on Perlmutter, as shown in Figure 12 (right). For a 7B parameter model, at 256 GPUs, PCCL underperforms NCCL with slowdown of 0.94 $\times$. However, as we scale to larger GPU counts, PCCL begins to outperform NCCL—achieving a 1.07 $\times$ speedup at 512 GPUs and a significantly higher 1.37 $\times$ speedup at 2048 GPUs.

Figure 13 presents batch times for strong scaling GPT3-1.3B on Frontier using PyTorch DDP. At smaller scales, RCCL outperforms PCCL, with PCCL showing a 0.55 $\times$ and 0.80 $\times$ slowdown at 128 and 256 GCDs, respectively. However, at higher GCD counts, PCCL rapidly closes this performance gap and ultimately surpasses RCCL, achieving a 1.8 $\times$ and 2.4 $\times$ speedup over RCCL at 1024 and 2048 GCDs, respectively.

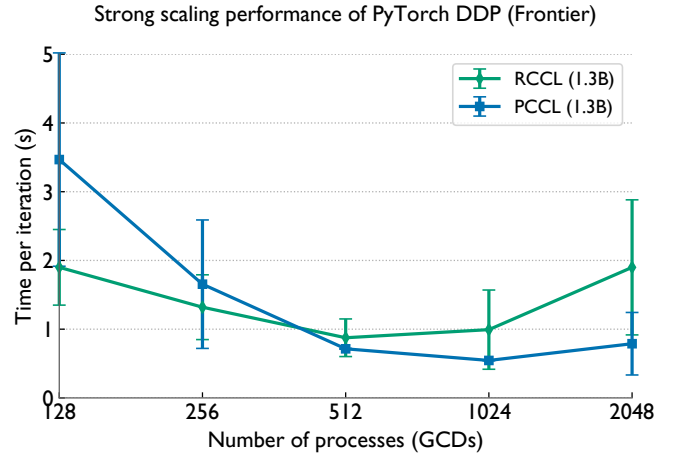


Fig. 13. Strong scaling performance of PyTorch DDP using RCCL and PCCL on Frontier for GPT-3 1.3B

All of these results highlight PCCL’s ability to deliver performance improvements for collective communication across multiple GPU architectures, and more importantly, translate those gains into meaningful end-to-end speedups for large-scale deep learning applications.

VII. RELATED WORK

Optimizing collective communication has been a long-standing challenge in high performance computing (HPC) and parallel computing and has been the focus on extensive research. Thakur et al.’s seminal work focuses on optimizing a plethora of collective operations including all-gathers and reduce-scatters in MPICH [9]. The authors explore the design space of several algorithms for each collective, and provide guidelines for selecting the most appropriate algorithm for different scenarios. In contrast, our work focuses on large-scale deep learning workloads and with messages sizes in the tens to hundreds of megabytes.

Patarsuk et al. propose the bandwidth-optimized ring algorithm for all-reduce operations [21]. Graham et al. develop optimize MPI collective to effectively exploit shared memory on multi-core systems. Chan et al. [22] develop highly optimized collectives for the IBM Blue Gene/L, exploiting unique properties of the system [23]. Kandalla et al. develop a scalable multi-leader hierarchical algorithm for all-gather [24]. Note that the focus of these works is on optimizing the performance of collective communication in traditional HPC workloads with small message sizes, and not on the large message sizes inherent to deep learning.

De Sensi et al. study the performance of NCCL, RCCL and Cray-MPICH across various state-of-the-art supercomputers across a variety of latency and bandwidth bound scenarios [10]. Cho et al. propose a multi-level hierarchical ring algorithm for all-reduce and study the tradeoff of bandwidth and latency between the flat and hierarchical ring algorithms [14]. In this work, we build on this and exploit more latency optimal

algorithms such as recursive doubling/halving in the inter-node levels of the hierarchy and also demonstrate how this design can be utilized to load-balance network traffic across NICs. Note that similar hierarchical designs have been explored in other works as well [25], [15].

Hidayetoglu et al. present HiCCL, a hierarchical collective communication library targeting the same systems as our work – Perlmutter and Frontier [26]. While HiCCL demonstrates performance advantages over NCCL and RCCL, these gains are primarily observed at smaller node counts. As shown in their evaluation, HiCCL’s performance matches or falls below vendor libraries at larger node counts. In contrast, PCCL is specifically designed and optimized for the large-scale regime, consistently outperforming vendor libraries at thousands of GPUs. Thus, HiCCL and PCCL address complementary performance regimes.

Cai et al. develop a systematic theoretical approach to synthesize novel communication algorithms for optimizing collective communication on a particular topology [27]. Cho et al. develop a strategy to maximize the overlap of a tree-based all-reduce with the computation in neural network training [28]. There is also a body of work focused on exploiting data compression to minimize communication overheads in distributed deep learning. For example, Feng et al. optimize all-to-all communication in recommendation model training via a novel error-bounded compression algorithm [29]. Huang et al. develop hZCCL, a communication library that enables collective operations on compressed data [30]. Zhou et al. develop a GPU-based compression scheme for all-gathers and reduce-scatters [31] and optimize FSDP [2] training at scale.

In the context of algorithm selection and optimization, Liu et al. develop parameterized ring and recursive algorithms for GPU-to-GPU collectives, and introduce a simulation technique for parameter autotuning [32]. Wilkins et al. develop ACLAiM, which uses machine learning for autotuning MPI collective communication [33], an approach conceptually related to our SVM-based adaptive dispatching mechanism.

Venkata et al. develop UCC, a unified collective communication library with pluggable transport layers supporting CPUs, GPUs, and DPUs, targeting broad applicability across programming models and hardware [34]. Si et al. describe NCCLX, Meta’s extension of NCCL with explicit topology awareness and multi-rail bandwidth aggregation, targeting collective communication at 100k+ GPU scale [35]. Similar to PCCL, NCCLX decomposes collectives hierarchically into intra-node and inter-node phases.

VIII. CONCLUSION

In this work, we investigated the current state of performance of collective communication routines in several libraries. Specifically, we focused on the all-gather, reduce-scatter, and all-reduce collectives, which find wide usage in several distributed training frameworks, and comprise the bulk of communication time in large-scale training runs. We evaluated the performance of Cray-MPICH, RCCL, and NCCL, three state-of-the-art communication libraries on HPE

Cray supercomputers, and highlighted shortcomings in their performance which make them unsuitable for scaling to large GPU counts.

We have developed PCCL, a new communication library with highly optimized implementations of all-gather, reduce-scatter, and all-reduce operations. PCCL leverages several optimizations designed to alleviate the performance bottlenecks of Cray-MPICH, NCCL, and RCCL, highlighted in this work. We demonstrated significant performance improvements over all three libraries, both in collective communication benchmarks as well as end-to-end training benchmarks. PCCL achieves substantial performance improvements, delivering 6–160 $\times$ speedups over RCCL and 28–70 $\times$ over Cray-MPICH for all-gather on 2048 GCDs of Frontier. These gains translate directly to end-to-end performance: in large-scale GPT-3-style training with DeepSpeed ZeRO-3, PCCL provides up to 3.6 $\times$ and 4.9 $\times$ speedups over RCCL for 7B and 13B parameter models on Frontier, respectively. For PyTorch DDP training, PCCL provides up to 2.4 $\times$ speedup over RCCL for a 1.3B parameter model on Frontier. In the future, we plan to benchmark PCCL on clusters with InfiniBand interconnects to evaluate its performance beyond Slingshot-based systems.

ACKNOWLEDGMENT

This material is based upon work supported in part by the National Science Foundation under Grant No. 2047120. We thank Joy Kitson of the Parallel Software and Systems Group for providing feedback on the paper and for improving the heatmap figures in the paper.

This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy (DOE) under Contract No. DE-AC05-00OR22725. This research also used resources of the National Energy Research Scientific Computing Center (NERSC), a U.S. DOE Office of Science User Facility, operated under Contract No. DE-AC02-05CH11231 using NERSC award DDR-ERCAP0034262.

REFERENCES

- [1] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimizations toward training trillion parameter models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’20. IEEE Press, 2020.
- [2] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, “Pytorch fsdp: Experiences on scaling fully sharded data parallel,” *Proc. VLDB Endow.*, vol. 16, no. 12, p. 3848–3860, aug 2023.
- [3] S. Singh and A. Bhatele, “AxiNN: An asynchronous, message-driven parallel framework for extreme-scale deep learning,” in *Proceedings of the IEEE International Parallel & Distributed Processing Symposium*, ser. IPDPS ’22. IEEE Computer Society, May 2022.
- [4] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, “Pytorch distributed: Experiences on accelerating data parallel training,” *Proc. VLDB Endow.*, vol. 13, no. 12, p. 3005–3018, Aug. 2020. [Online]. Available: <https://doi.org/10.14778/3415478.3415530>
- [5] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” Tech. Rep., 2020.

- [6] P. Goyal, P. Dollár, R. B. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, “Accurate, large minibatch SGD: training imagenet in 1 hour,” *CoRR*, vol. abs/1706.02677, 2017. [Online]. Available: <http://arxiv.org/abs/1706.02677>
- [7] S. Singh, P. Singhanian, A. Ranjan, J. Kirchenbauer, J. Geiping, Y. Wen, N. Jain, A. Hans, M. Shu, A. Tomar, T. Goldstein, and A. Bhatele, “Democratizing AI: Open-source scalable LLM training on GPU-based supercomputers,” in *Proceedings of the ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’24, Nov. 2024.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [9] R. Thakur and W. D. Gropp, “Improving the performance of collective operations in mpich,” in *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, J. Dongarra, D. Laforenza, and S. Orlando, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 257–267.
- [10] D. De Sensi, L. Pichetti, F. Vella, T. De Matteis, Z. Ren, L. Fusco, M. Turisini, D. Cesarini, K. Lust, A. Trivedi, D. Roweth, F. Spiga, S. Di Girolamo, and T. Hoefler, “Exploring gpu-to-gpu communication: Insights into supercomputer interconnects,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, ser. SC ’24. IEEE Press, 2024. [Online]. Available: <https://doi.org/10.1109/SC41406.2024.00039>
- [11] H. P. Enterprise, *HPE Cassini Performance Counters*, 2024, accessed: 2025-02-24. [Online]. Available: https://cpe.ext.hpe.com/docs/latest/getting_started/HPE-Cassini-Performance-Counters.html
- [12] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, “Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.04786>
- [13] S. Jeaugey, “Pat: a new algorithm for all-gather and reduce-scatter operations at scale,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.20252>
- [14] M. Cho, U. Finkler, D. Kung, and H. Hunter, “Blueconnect: Decomposing all-reduce for deep learning on heterogeneous network hierarchy,” in *Proceedings of Machine Learning and Systems*, A. Talwalkar, V. Smith, and M. Zaharia, Eds., vol. 1, 2019, pp. 241–251. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/file/0c8abcf158ed12d0dd94480681186fda-Paper.pdf
- [15] T. Thao Nguyen, M. Wahib, and R. Takano, “Hierarchical Distributed-Memory Multi-Leader MPI-Allreduce for Deep Learning Workloads,” in *2018 Sixth International Symposium on Computing and Networking Workshops (CANDARW)*. Los Alamitos, CA, USA: IEEE Computer Society, Nov. 2018, pp. 216–222. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CANDARW.2018.00048>
- [16] J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. R. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein, “Scaling up test-time compute with latent reasoning: A recurrent depth approach,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.05171>
- [17] A. Singh, “Load-balanced routing in interconnection networks,” Ph.D. dissertation, Dept. of Electrical Engineering, Stanford University, 2005, http://cva.stanford.edu/publications/2005/thesis_arjuns.pdf.
- [18] T. B. Brown *et al.*, “Language models are few-shot learners,” *CoRR*, vol. abs/2005.14165, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [19] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “Opt: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [20] A. Gokaslan and V. Cohen, “Openwebtext corpus,” <http://Skylion007.github.io/OpenWebTextCorpus>, 2019.
- [21] P. Patarasuk and X. Yuan, “Bandwidth optimal all-reduce algorithms for clusters of workstations,” *J. Parallel Distrib. Comput.*, vol. 69, no. 2, p. 117–124, feb 2009. [Online]. Available: <https://doi.org/10.1016/j.jpdc.2008.09.002>
- [22] E. Chan, R. van de Geijn, W. Gropp, and R. Thakur, “Collective communication on architectures that support simultaneous communication over multiple links,” in *Proceedings of the Eleventh ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP ’06. New York, NY, USA: Association for Computing Machinery, 2006, p. 2–11. [Online]. Available: <https://doi.org/10.1145/1122971.1122975>
- [23] R. L. Graham and G. Shipman, “Mpi support for multi-core architectures: Optimized shared memory collectives,” in *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, A. Lastovetsky, T. Kechadi, and J. Dongarra, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 130–140.
- [24] K. Kandalla, H. Subramoni, G. Santhanaraman, M. Koop, and D. K. Panda, “Designing multi-leader-based allgather algorithms for multi-core clusters,” in *2009 IEEE International Symposium on Parallel and Distributed Processing*, 2009, pp. 1–8.
- [25] J. M. Hashmi, S. Chakraborty, M. Bayatpour, H. Subramoni, and D. K. Panda, “Designing efficient shared address space reduction collectives for multi-/many-cores,” in *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018, pp. 1020–1029.
- [26] M. Hidayetoglu, S. G. de Gonzalo, E. Slaughter, P. Surana, W. mei Hwu, W. Gropp, and A. Aiken, “Hiccl: A hierarchical collective communication library,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.05962>
- [27] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, “Synthesizing optimal collective algorithms,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 62–75. [Online]. Available: <https://doi.org/10.1145/3437801.3441620>
- [28] S. Cho, H. Son, and J. Kim, “Logical/physical topology-aware collective communication in deep learning training,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 56–68.
- [29] H. Feng, B. Zhang, F. Ye, M. Si, C.-H. Chu, J. Tian, C. Yin, S. Deng, Y. Hao, P. Balaji, T. Geng, and D. Tao, “Accelerating communication in deep learning recommendation model training with dual-level adaptive lossy compression,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.04272>
- [30] J. Huang, S. Di, X. Yu, Y. Zhai, J. Liu, Z. Jian, X. Liang, K. Zhao, X. Lu, Z. Chen, F. Cappello, Y. Guo, and R. Thakur, “hzcl: Accelerating collective communication with co-designed homomorphic compression,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, ser. SC ’24. IEEE Press, 2024. [Online]. Available: <https://doi.org/10.1109/SC41406.2024.00110>
- [31] Q. Zhou, Q. Anthony, L. Xu, A. Shafi, M. Abduljabbar, H. Subramoni, and D. K. D. Panda, “Accelerating distributed deep learning training with compression assisted allgather and reduce-scatter communication,” pp. 134–144, 2023.
- [32] P. Liu, S. Rhee, M. Wilkins, and P. Dinda, “Parameterized algorithms and parameter selection for fast gpu-gpu collective communication,” in *2025 33rd International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2025, pp. 1–9.
- [33] M. Wilkins, Y. Guo, R. Thakur, P. A. Dinda, and N. Hardavellas, “Acclaim: Advancing the practicality of mpi collective communication autotuning using machine learning,” *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 161–171, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:251793247>
- [34] M. G. Venkata, V. Petrov, S. Lebedev, D. Bureddy, F. Aderholdt, J. Ladd, G. Bloch, M. Dubman, and G. Shainer, “Unified collective communication (UCC): a unified library for cpu, gpu, and DPU collectives,” in *IEEE Symposium on High-Performance Interconnects, HOTI 2024, Albuquerque, NM, USA, August 21–23, 2024*. IEEE, 2024, pp. 37–46. [Online]. Available: <https://doi.org/10.1109/HOTI63208.2024.00018>
- [35] M. Si, P. Balaji, Y. Chen, C.-H. Chu, A. Gangidi, S. Hasan, S. Iyengar, D. Johnson, B. Liu, R. Ren, D. Shah, A. J. Shetty, G. Steinbrecher, Y. Wang, B. Wu, X. Xie, J. Yang, M. Yang, K. Yu, M. Yu, C. Zhao, W. Bland, D. Boyda, S. Gumudavelli, P. Kannan, C. Lumezanu, R. Miao, Z. Qu, V. Ramesh, M. Samoylov, J. Seidel, S. Sundaresan, F. Tian, Q. Tan, S. Zhang, Y. Zhao, S. Zheng, A. Zhu, and H. Zeng, “Collective communication for 100k+ gpus,” 2026. [Online]. Available: <https://arxiv.org/abs/2510.20171>